AI for Economic Research: Dynamic Models, Language, and Agents
Lecture 7.1b: Word Embeddings

Zhigang Feng

Spring 2026

Topic 7.1b Agenda

1. **Word Embeddings** — TF-IDF → Word2Vec → GloVe
2. **Geometry of meaning** — Arrow–Debreu analogy, analogies and bias
3. **From static to contextual** — limits of static embeddings; macro applications

Builds on Topic 7.1a (tokenization). Bridge to Topic 7.2 (self-attention and the Transformer).

Arc: T1a Why+Tokens → T1b Embeddings → T2a Attention → T2b Transformer → T3a Training → T3b Limits → T4 Applications.

Word Embeddings

Why Embed Tokens into Vectors?

- Neural networks operate on real-valued vectors, not discrete symbols.
- An **embedding** is a learned function

$$\mathcal{E}\colon \{1, \ldots, M\} \longrightarrow \mathbb{R}^d,$$

  mapping each token ID to a dense vector of dimension d (typically $d = 128$ to 4096).
- **Think of it as a basis problem from linear algebra.** We want to find a coordinate system — a set of d orthogonal "semantic axes" (e.g. monetary-policy stance, positive valence, corporate context) — such that every token maps to a point in $\mathbb{R}^d$. Once tokens are coordinates, we can do arithmetic: measure distances, compute cosine similarities, add and subtract meaning. We will later connect this to Arrow–Debreu securities in financial markets. The challenge: we do not know the right axes in advance. They must be *learned* from data.
- **What we want from embeddings:**
  - Semantically similar tokens should have nearby vectors (close coordinates in the learned basis)
  - Algebraic operations (sums, differences) should reflect semantic relations
  - The mapping should be *learned* from data, not hand-designed
- **Bonus:** we will also need a *positional* signal so the model knows token order—more on this when

Three Approaches to Word Representation

The problem from the previous slide: find a mapping from tokens to vectors that captures semantic relationships. Three generations of methods have tried to solve this, each fixing a specific failure of its predecessor. Understanding this progression explains *why* the Transformer was needed.

Method	Type	Idea	Year
TF-IDF	Sparse, frequency	Re-weight word counts	1970s
Word2Vec	Dense, predictive	Predict neighbors in window	2013
GloVe	Dense, statistical	Factor co-occurrence matrix	2014
Transformer embeddings	Dense, contextual	Vary by surrounding tokens	2017+

Transformer row previewed for completeness; full treatment in T2a/T2b. This topic covers only the first three rows.

- TF-IDF: a bag-of-words feature. Transparent and easy to interpret.
- Word2Vec / GloVe: produce a single vector per word *type* (*static* embeddings).
- Transformer embeddings: produce a different vector for each *occurrence* of a word, depending on

Term Frequency (TF)

- **Definition:** share of document d consisting of word w:

$$\mathrm{TF}(w, d) \;=\; \frac{\text{count}(w, d)}{\text{total words in } d}.$$

- **Toy example.** FOMC-like sentence d: *"the Fed raised rates because inflation persisted"* (7 words). Every word appears once, so TF is $1/7 \approx 0.14$ for each:

word	count in d	$\mathrm{TF}(w, d)$
the	1	0.14
Fed	1	0.14
raised	1	0.14
rates	1	0.14
inflation	1	0.14

- **Limitation:** TF rewards *frequent* words. Across a corpus, function words (*the, a, is*) dominate even though they carry almost no semantic signal. We need a counterweight.

Inverse Document Frequency (IDF)

- **Definition:** how rare word w is across a collection of N documents:

$$\text{IDF}(w) \;=\; \log\!\left(\frac{N}{\#\{\text{docs containing } w\}}\right).$$

- **Toy example.** Corpus of $N = 100$ FOMC speeches:

word	docs containing it	IDF(w)
the	100	$\log(100/100) = 0$
inflation	60	$\log(100/60) \approx 0.51$
yield curve	5	$\log(100/5) \approx 3.00$

- **Intuition:** rare $\Rightarrow$ informative $\Rightarrow$ up-weighted. *"the"* gets 0 automatically — no stop-word list needed. *"yield curve"* gets a large multiplier because it distinguishes a few speeches from the rest.

Putting It Together: TF-IDF

- **Product formula:** each (word, document) pair gets a weighted score

$$\text{TF-IDF}(w, d) \;=\; \text{TF}(w, d) \cdot \text{IDF}(w).$$

  Stacking across all words in vocabulary $V \Rightarrow$ each document is a sparse vector in $\mathbb{R}^{|V|}$.

- **Toy 3-speech matrix** (illustrative scores):

	the	*inflation*	*yield curve*
speech A (hawkish)	0	0.08	0.00
speech B (dovish)	0	0.05	0.00
speech C (technical)	0	0.02	0.21

- **Properties:**
  - Down-weights common words automatically (no stop-word list)
  - Highlights distinctive words for each document
  - Lives in $\mathbb{R}^{|V|}$ — very high-dimensional, very sparse
  - *No* semantic similarity: *"central bank"* and *"Federal Reserve"* are orthogonal $\Rightarrow$ motivation for dense embeddings (next slide).

Word2Vec: Dense Embeddings from Context

- **Distributional hypothesis** (Firth, 1957): *"You shall know a word by the company it keeps."*
- **Idea** (Mikolov et al., 2013): for each word in a corpus, look at a window of $\pm L$ neighboring words. Learn a vector for the center word that *predicts* its neighbors (*Skip-gram*) or learn one that *is predicted by* its neighbors (*CBOW*).
- **Training output:** a single static vector $\mathbf{u}_w \in \mathbb{R}^d$ for each word w in the vocabulary, with d typically 100–300.
- **Geometric magic:** after training, simple vector arithmetic captures analogies, e.g.

$$\mathbf{u}_{\text{king}} - \mathbf{u}_{\text{man}} + \mathbf{u}_{\text{woman}} \approx \mathbf{u}_{\text{queen}}.$$

  Words living on the same semantic axis differ by approximately the same vector.

Skip-Gram in One Slide: The Math

- Each word w has *two* learned vectors: an input vector $\mathbf{u}_w$ and an output vector $\mathbf{v}_w$ (often merged after training).
- For a center word w_t and a context word w_c in its window, the model predicts

$$P\big(w_c \mid w_t\big) \;=\; \frac{\exp\big(\mathbf{u}_{w_t}^{\top}\mathbf{v}_{w_c}\big)}{\sum_{j=1}^{M}\exp\big(\mathbf{u}_{w_t}^{\top}\mathbf{v}_{w_j}\big)}.$$

- Training objective (negative log-likelihood over all (center, context) pairs):

$$\mathcal{L} \;=\; -\sum_{t}\;\sum_{c\in\mathsf{window}(t)} \log P\big(w_c \mid w_t\big).$$

- Optimized with stochastic gradient descent (in practice with *negative sampling* to avoid the full vocabulary softmax).
- **Result:** words with similar context distributions end up with similar $\mathbf{u}$ vectors.

Skip-Gram: Walkthrough on "central banks target inflation"

- **Sentence:** *"central banks target inflation"*, window radius $L = 2$, center word = *target*.
- Training pairs formed from this center word:

$$(\textit{target}, \textit{central}), \quad (\textit{target}, \textit{banks}), \quad (\textit{target}, \textit{inflation}).$$

- For each pair, the score is the dot product $\mathbf{u}_{\text{target}}^{\top}\mathbf{v}_{\text{context}}$. Gradient descent *increases* this score for observed pairs and *decreases* it for random (negative) pairs.
- **Across the whole corpus:** *target* appears near *rates*, *inflation*, *policy*, *Fed*. So do *raise* and *cut*. Consequently

$$\mathbf{u}_{\text{target}} \text{ ends up close to } \mathbf{u}_{\text{raise}},\ \mathbf{u}_{\text{cut}} \text{ and far from } \mathbf{u}_{\text{apple}}.$$

- **Takeaway:** analogies emerge because consistent co-occurrences place related words at similar positions in $\mathbb{R}^d$.

GloVe: Global Co-occurrence Statistics

- Pennington et al. (2014): instead of sliding-window prediction, factor a *global* matrix.
- Let X_{ij} count how often word j appears near word i across the entire corpus. GloVe fits embeddings to satisfy

$$\mathbf{u}_{w_i}^{\top}\,\mathbf{v}_{w_j} + b_i + b_j \;\approx\; \log X_{ij}.$$

- **Toy counts on a central-bank corpus** X_{ij} (illustrative):

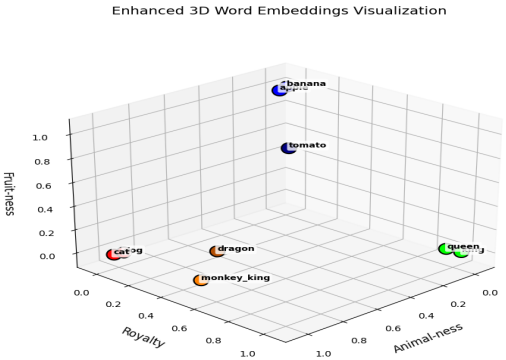
	rates	*inflation*	*apple*
Fed	820	410	3
ECB	540	390	2
hawkish	210	160	0

  $\log X_{\text{Fed,rates}} \approx 6.7$ (large target dot-product) vs. $\log X_{\text{Fed,apple}} \approx 1.1$ (small). Fitting the equation therefore forces $\mathbf{u}_{\text{Fed}}$ close to $\mathbf{v}_{\text{rates}}, \mathbf{v}_{\text{inflation}}$ and orthogonal to $\mathbf{v}_{\text{apple}}$ — monetary-policy structure falls out of corpus-wide statistics.

- **Contrast with Word2Vec:** predictive / local / streaming (W2V) vs. statistical / global / one-shot factorization (GloVe). Both produce *static* dense vectors with similar geometry. Pick W2V for massive streaming corpora, GloVe when a full co-occurrence matrix is cheap to build

Geometry of Meaning

A 3D Toy of Word Embeddings



The three axes loosely correspond to "Animal-ness", "Royalty", and "Fruit-ness". Words like *cat*, *king*, and *apple* cluster along the dimension that fits them best. Real LLM embeddings live in hundreds of dimensions, but the same intuition holds.

Analogy: Embeddings as Arrow–Debreu Securities

- A d-dimensional embedding is the linguistic cousin of an Arrow–Debreu payoff vector: the d coordinates are payoffs across d latent semantic "states."
- **Dictionary, term by term:**
  - *d-dim embedding* $\equiv$ payoff vector across d latent semantic states (royalty, monetary-policy stance, animal-ness, ...).
  - *Inner product* $\mathbf{u}^\top \mathbf{v} \equiv$ covariance of state-payoffs; *orthogonality* $\Leftrightarrow$ uncorrelated semantic content.
  - *Cosine similarity* $\cos(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u}^\top \mathbf{v}}{\|\mathbf{u}\| \, \|\mathbf{v}\|} \equiv$ correlation under unit-norm payoffs — length normalization removes the influence of an asset's overall scale.
  - *Arrow–Debreu prices over states* $\leftrightarrow$ *softmax attention weights over tokens* (preview of T2a) — both are non-negative weights summing to 1 over a state space, and both price/aggregate state-contingent payoffs.
- **The pedagogical pay-off.** This is why economists already have an intuition for everything in T2a. Self-attention is just a learned, content-addressed state-pricing kernel.

Analogies and the Bias Problem

- **Geometric magic** (Mikolov et al., 2013): simple vector arithmetic captures analogies.

$$\mathbf{u}_{\text{king}} - \mathbf{u}_{\text{man}} + \mathbf{u}_{\text{woman}} \approx \mathbf{u}_{\text{queen}}$$

$$\mathbf{u}_{\text{Paris}} - \mathbf{u}_{\text{France}} + \mathbf{u}_{\text{Italy}} \approx \mathbf{u}_{\text{Rome}}$$

- Words living on the same semantic axis differ by approximately the same vector — gender, capital city, comparative tense, monetary stance.
- **Bias is encoded too** (Bolukbasi et al., 2016, *NeurIPS*; Caliskan, Bryson & Narayanan, 2017, *Science*):
  - $\mathbf{u}_{\text{man}} - \mathbf{u}_{\text{woman}}$ has a large component on *programmer* vs. *homemaker*.
  - Word embeddings reproduce IAT-style biases at human magnitudes.
  - For economic text: occupation/gender, race/credit-risk, region/sentiment correlations all transfer from the training corpus into the geometry.
- **Lesson:** embeddings are a mirror of the corpus. Validating that a sentiment vector reflects *policy* stance and not *speaker identity* is a real research step (returns in Topic 7.4).

Limits of Static Embeddings

What static embeddings miss

- **Polysemy:** "rate" = interest rate? exchange rate? growth rate? One vector for all senses.
- **Negation:** "GDP growth NOT slowing" and "GDP growth slowing" have nearly identical mean vectors.
- **Long-range context:** fixed window misses dependencies across paragraphs.
- **Corpus dependence:** "mask" before vs. after 2020 are very different concepts.

What we need next

- Vectors that change with *context*, not just word identity
- A mechanism for tokens to "look at" other tokens, near or far
- A way to learn arbitrary relational patterns, not just co-occurrence within a window

$\Longrightarrow$ **The Transformer.**

Macro Applications

From Embeddings to Macro Variables

- Once a sentence (or document) is a vector, every downstream task is a familiar one:
  - **Classification:** hawkish / dovish / neutral $\Rightarrow$ logistic regression on the embedding.
  - **Regression:** project the embedding onto a numeric outcome (sentiment score, uncertainty index).
  - **Clustering / topics:** k-means or LDA on embeddings $\Rightarrow$ discover policy themes.
  - **Retrieval:** nearest neighbor under cosine $\Rightarrow$ "find me speeches like this one" (Lecture 8).

- **Three concrete patterns from the literature:**
  1. **Signal** — FOMC sentiment series predicting yields (Hansen & McMahon, 2016, *JIE*).
  2. **Measurement** — ECB Word2Prices and central-bank communication tone (Acosta & Meade, 2015; Schnabel et al., 2024).
  3. **Generation** — LLM-summarized 10-K risk sections feeding a credit-spread regression.

- **Topic 7.4** goes deep on each — corpora (FOMC, SEC EDGAR, BIS speeches), domain models (FinBERT, CentralBankRoBERTa), validation, and pitfalls.

Macro Application Gallery (Quick Tour)

Application	Representation	Outcome / use
EPU index (Baker–Bloom–Davis, 2016, *QJE*)	Keyword counts	Monthly index, VAR shock
FOMC transparency (Hansen, McMahon & Prat, 2018, *QJE*)	LDA topics over transcripts	How disclosure regime changed speech
News-based recession (Bybee, Kelly, Manela & Xiu, 2024, *J. Finance*)	Topic shares from news	Recession nowcast vs. surveys
FOMC sentiment (Hansen & McMahon, 2016)	Dictionary + Word2Vec	Predict Treasury yields, macro surprises
ECB Word2Prices	Static + contextual emb.	Map speeches to inflation expectations
FinBERT / CentralBankRoBERTa	Domain-tuned BERT contextual	Sentence-level stance on filings, minutes

Common pattern: (1) tokenize → (2) embed → (3) aggregate to document/series → (4) regress / forecast / classify. Steps (1)–(2) are *this topic and its predecessor*; (3) is a research design choice; (4) is standard econometrics.

Wrap-up: From Tokens to Transformers

What we covered.

1. **Three roles of text:** signal, measurement, generation (Topic 7.1a).
2. **Tokenization:** BPE, WordPiece, SentencePiece. Subwords beat words and characters; tokenizer ships with the model (Topic 7.1a).
3. **From counts to vectors:** TF-IDF → Word2Vec / GloVe (static) → contextual embeddings.
4. **Geometry:** cosine similarity, analogies, bias — all read off the embedding space.
5. **Macro:** every downstream task starts from a sentence vector; Topic 7.4 deep-dives.

What's next: Topic 7.2a — Attention Mechanism, then 7.2b — Transformer Architecture.

- Static embeddings can't see context, can't handle polysemy, can't combine across long ranges.
- The Transformer fixes all three with one mechanism: *self-attention* — a learned, content-addressable lookup over the entire sequence.
- Then: T3a unpacks training (pre-train → fine-tune → alignment); T3b audits limits; T4 surveys economic applications.

Tokens become vectors. Vectors become geometry. Geometry becomes economics.